

Data Structures and Algorithms

TREE Data Structure

LECTURE 5

Dr.Shada Mabgar

INTRODUCTION

- **Non-linear** data structures are used to represent the data containing **hierarchical or network relationship** between the elements.
- **Trees and graphs** are examples of non-linear data structures.
- In **non-linear data structures**, every data element may have more than one predecessor as well as successor.
- **Elements do not form any particular linear sequence.**

Introduction to Tree Data Structure

- **Tree Data Structure** is a **non-linear** data structure in which a **collection of elements** known as **nodes** are **connected** to each other via **edges** such that there exists exactly one path between any two nodes.

Why Trees?

1. One reason to use trees might be because you want to **store information** that naturally forms a **hierarchy**. For example, **the file system on a computer** .
2. **Trees** (with some ordering e.g., BST) provide moderate access/search (quicker than Linked List and slower than arrays).
3. **Trees** provide moderate **insertion/deletion** (quicker than Arrays and slower than Unordered Linked Lists).

Applications

Main applications of trees include:

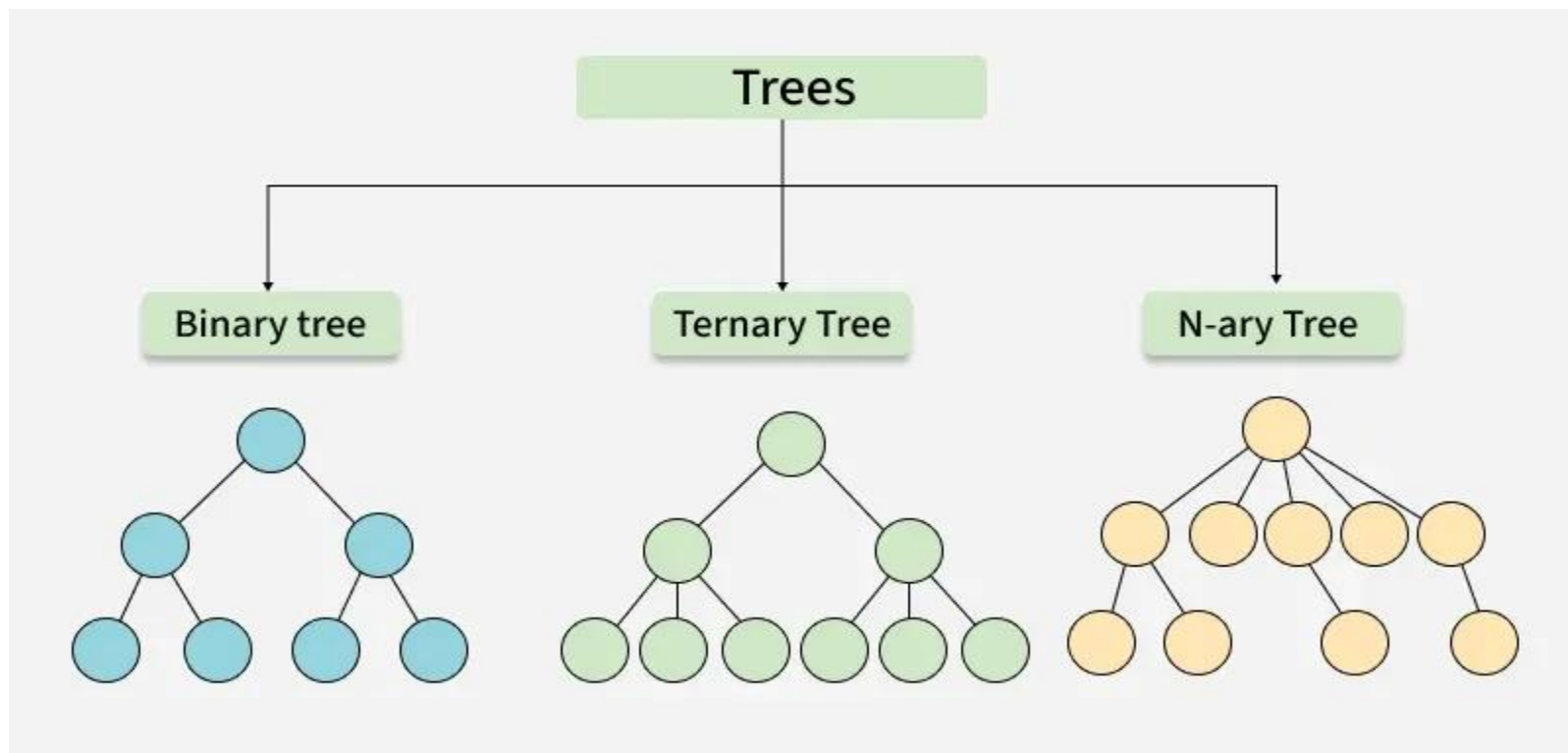
1. Manipulate hierarchical data.
2. Make information easy to search.
3. Manipulate sorted lists of data.
4. In chess game to store defense moves of player.
5. Router Algorithms
6. In Computer Graphics.
7. To evaluate an expression.
8. Form of a multi-stage decision.

Technical Definition

- A tree is a collection of **nodes**.
- Nodes are connected by **edges**.
- Each node contains a **value** or **data**, and it may or may not have a child node .
- The first node of the tree is called the **root**.
- If this root node is connected by another node, the root is then a **parent** node and the connected node is a **child**.
- In a tree data structure, if we have **N** number of nodes then we can have a maximum of **N-1** number of links.

Types of Tree Data Structures

- **Tree data structures** can be classified based on the **number of children** each **node** can have.

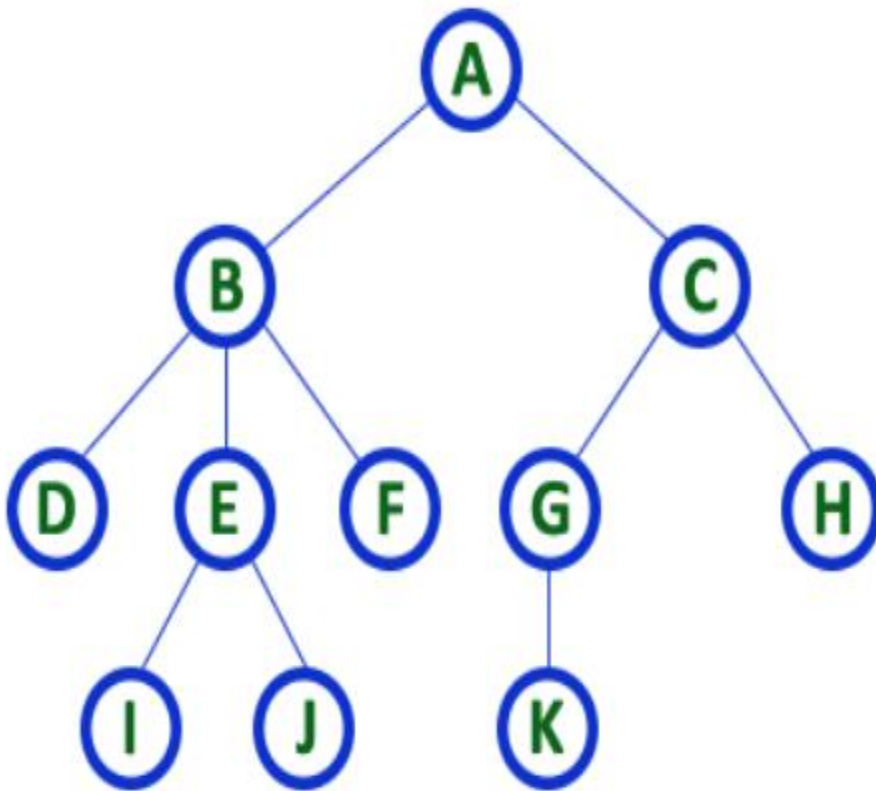


Types of Tree Data Structures

- **Binary Tree:** Each **node** can have a **maximum of two children**.
- **Ternary Tree:** Each **node** can have at most **three children**, often **labeled** as **left**, **middle**, and **right**.
- **N-ary Tree (or Generic Tree):** Each **node** can have any **number of children**, each node typically contains: Some data, A list of references to its **children (duplicates are not allowed)**.

General Tree

Example :

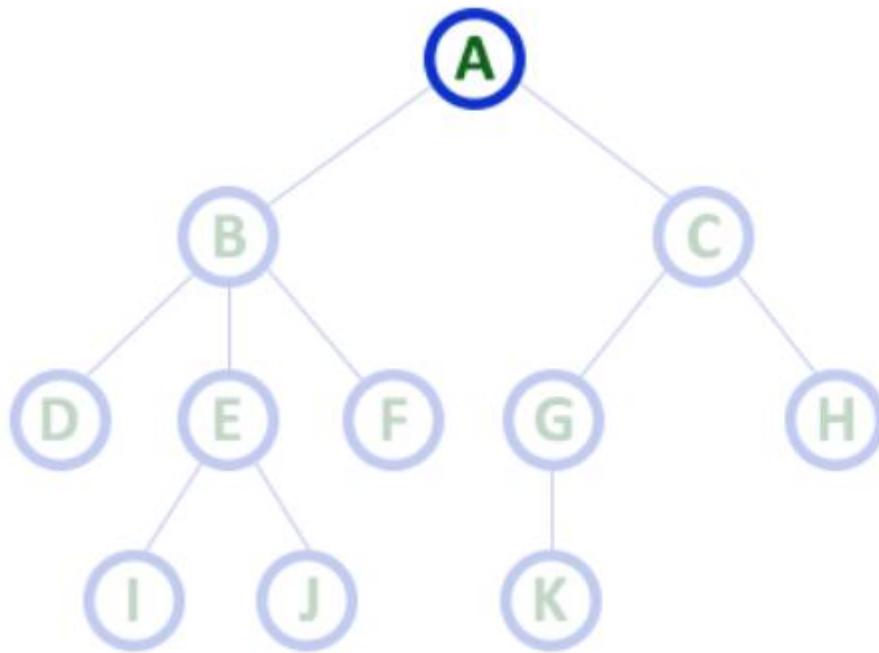


TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

Terminologies In Tree Data Structure

1. Root is the topmost node of the tree, is the origin of the tree data structure .

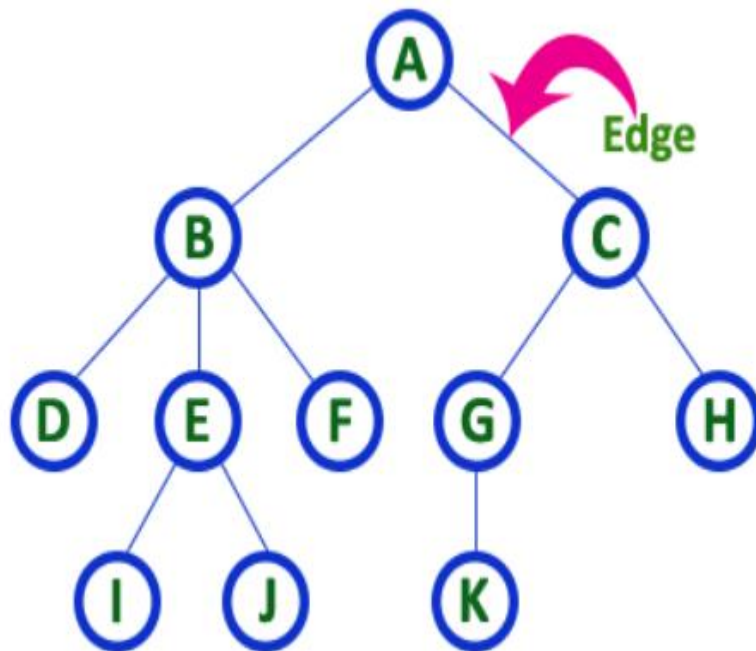


Here 'A' is the 'root' node

- In any tree the first node is called as ROOT node

Terminologies In Tree Data Structure

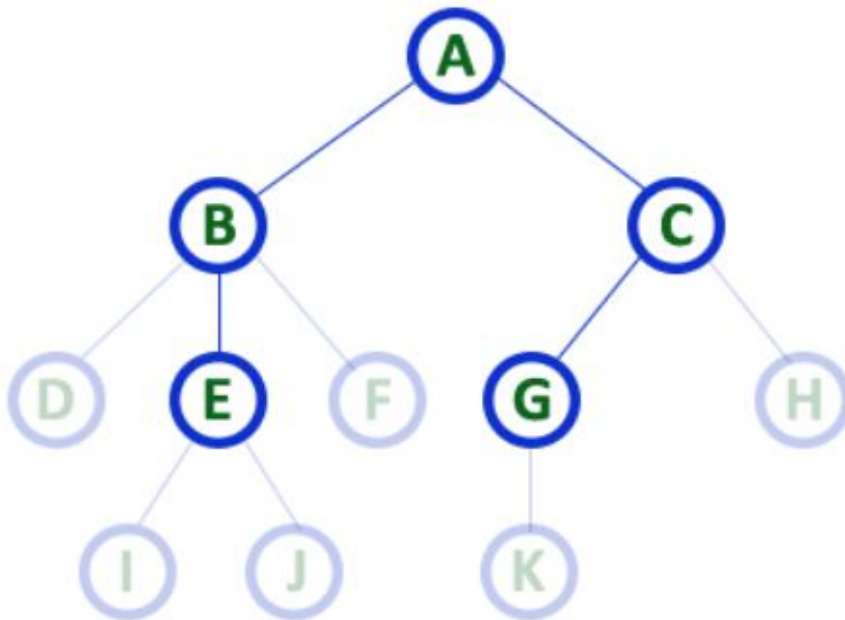
2. Edge is the link between two nodes . In a tree with 'N' number of nodes there will be a maximum of 'N-1' number of edges.



- In any tree, 'Edge' is a connecting link between two nodes.

Terminologies In Tree Data Structure

3. Parent, the node which has a branch from it to any other node is called a **parent node**. **Parent node** can also be defined as "The node which has **child / children**".

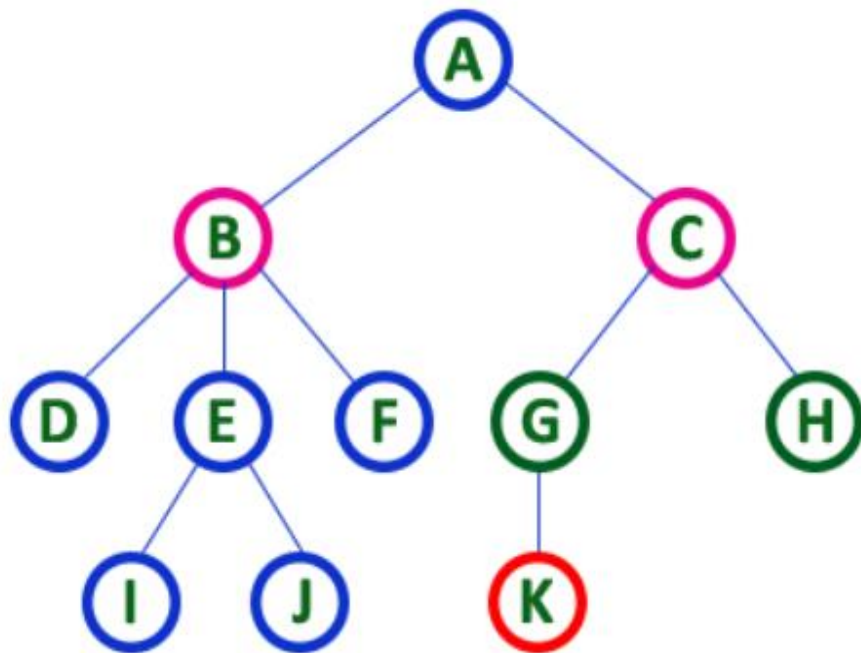


Here A, B, C, E & G are **Parent** nodes

- In any tree the node which has child / children is called '**Parent**'
- A node which is predecessor of any other node is called '**Parent**'

Terminologies In Tree Data Structure

4. **Child** is a node that has a parent node .



Here **B & C** are **Children** of **A**

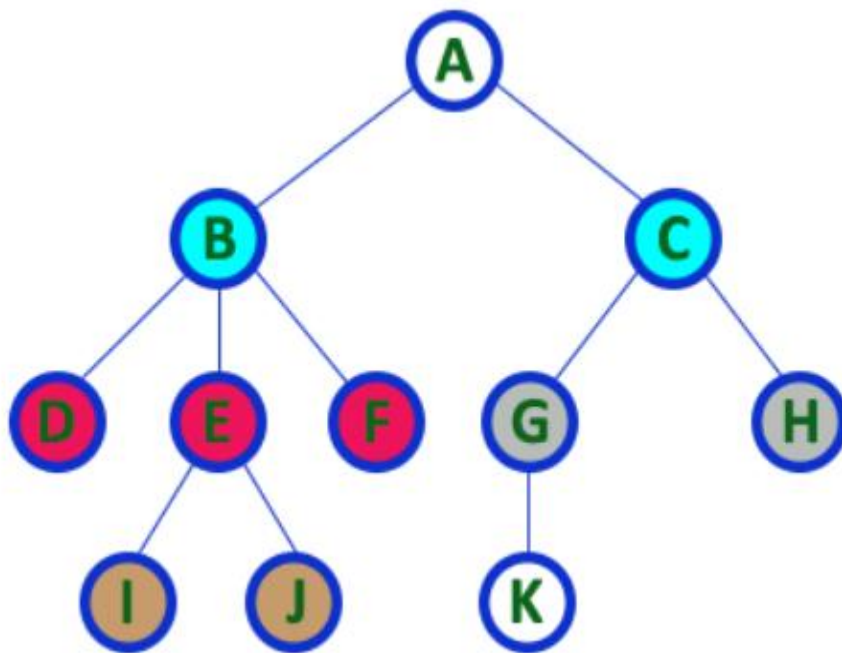
Here **G & H** are **Children** of **C**

Here **K** is **Child** of **G**

- descendant of any node is called as **CHILD Node**

Terminologies In Tree Data Structure

5. Siblings the nodes with the same parent are called **Sibling nodes**.



Here **B & C** are **Siblings**

Here **D E & F** are **Siblings**

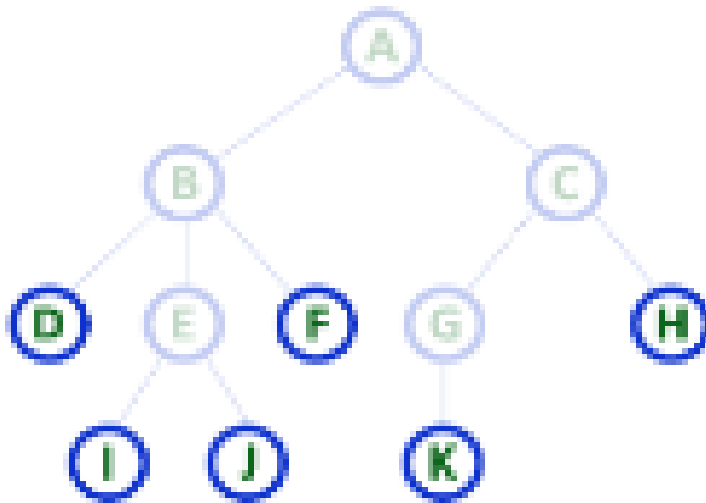
Here **G & H** are **Siblings**

Here **I & J** are **Siblings**

- In any tree the nodes which has same Parent are called '**Siblings**'
- The children of a Parent are called '**Siblings**'

Terminologies In Tree Data Structure

6. Leaf is a node that does not have a child node in the tree .In a tree data structure, the leaf nodes are also called as **External Nodes**

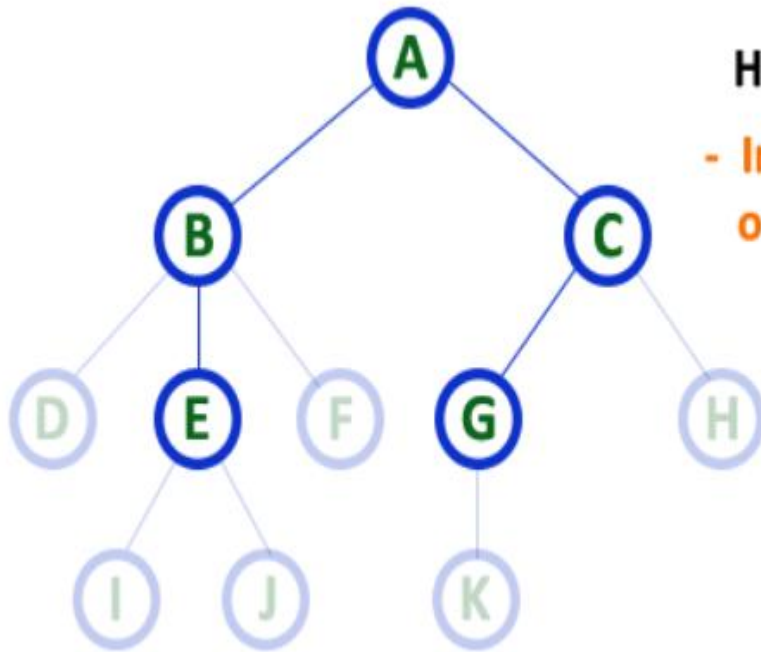


Here D, I, J, F, K & H are **Leaf** nodes

- In any tree the node which does not have children is called '**Leaf**'
- A node without successors is called a '**leaf**' node

Terminologies In Tree Data Structure

7. Internal Nodes : the node which has at least one child .



Here A, B, C, E & G are **Internal** nodes

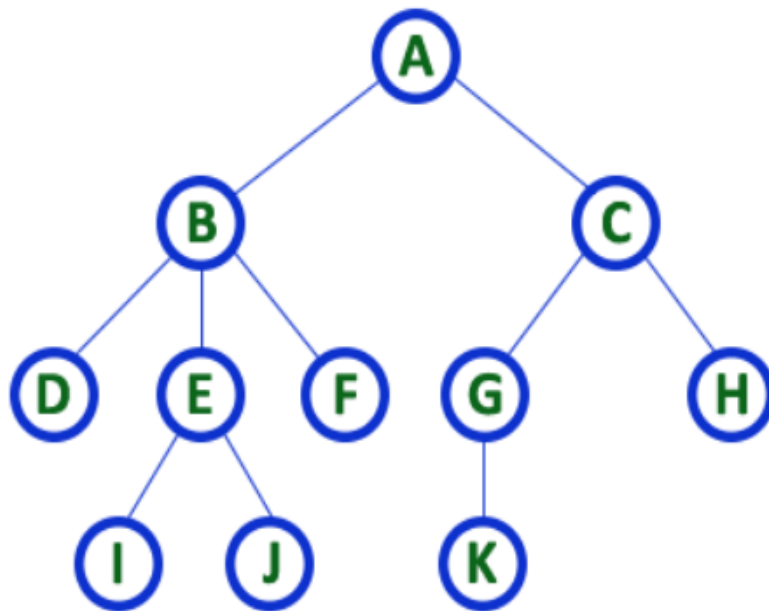
- In any tree the node which has atleast one child is called '**Internal**' node

- Every non-leaf node is called as '**Internal**' node

Terminologies In Tree Data Structure

8. Degree :the total number of children of a node

The highest degree of a node among all the nodes in a tree is called as '**Degree of Tree**'.



Here **Degree** of B is 3

Here **Degree** of A is 2

Here **Degree** of F is 0

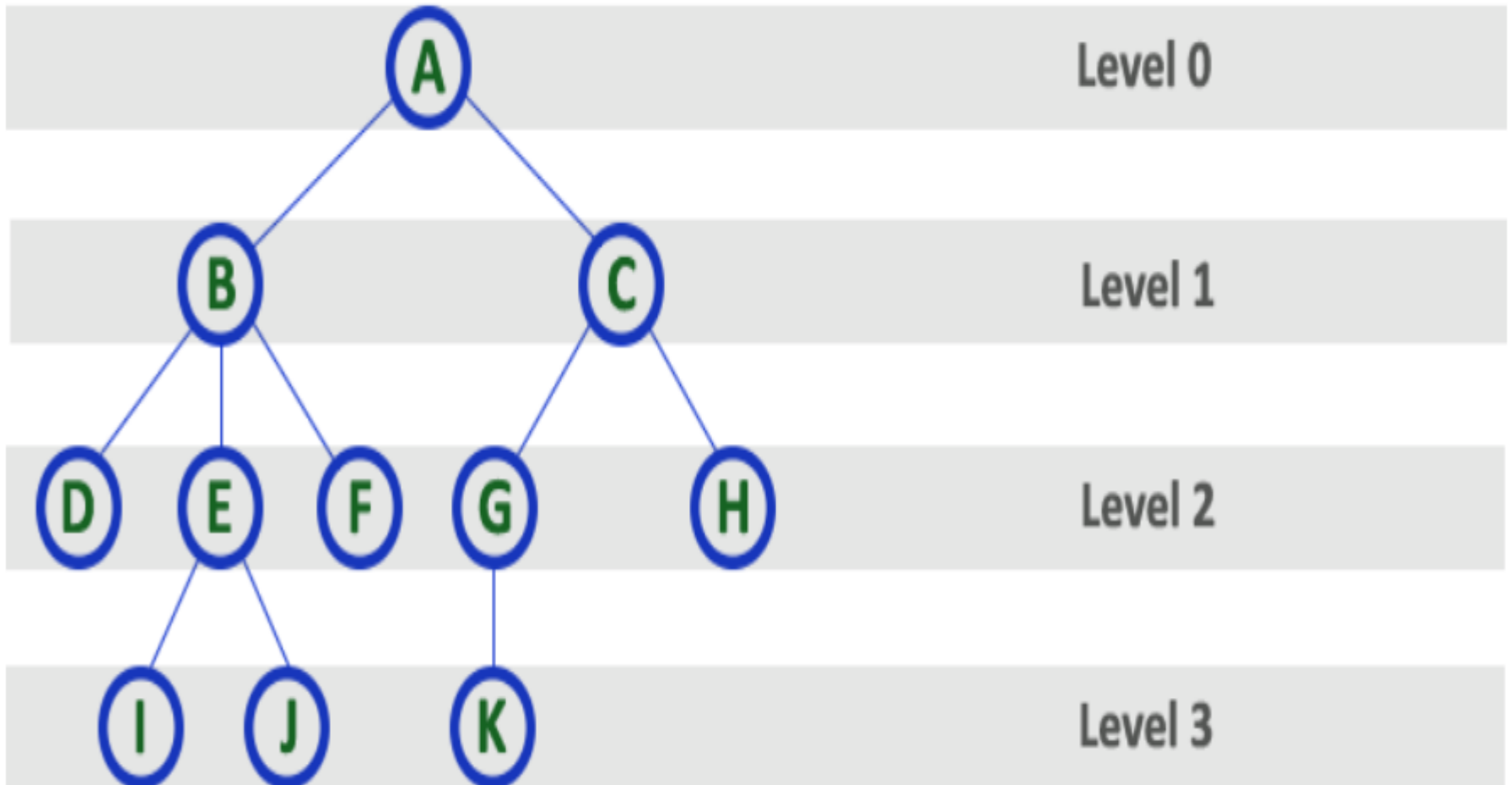
- In any tree, '**Degree**' of a node is total number of children it has.

Terminologies In Tree Data Structure

- 9. Level :** the root node is said to be at **Level 0** and the children of root node are at Level 1 and the children of the nodes which are at **Level 1** will be at **Level 2** and so on...
- In simple **words**, in a tree each step from **top** to **bottom** is called as a **Level** and the Level count starts with '**0**' and **incremented** by **one** at each level (Step).

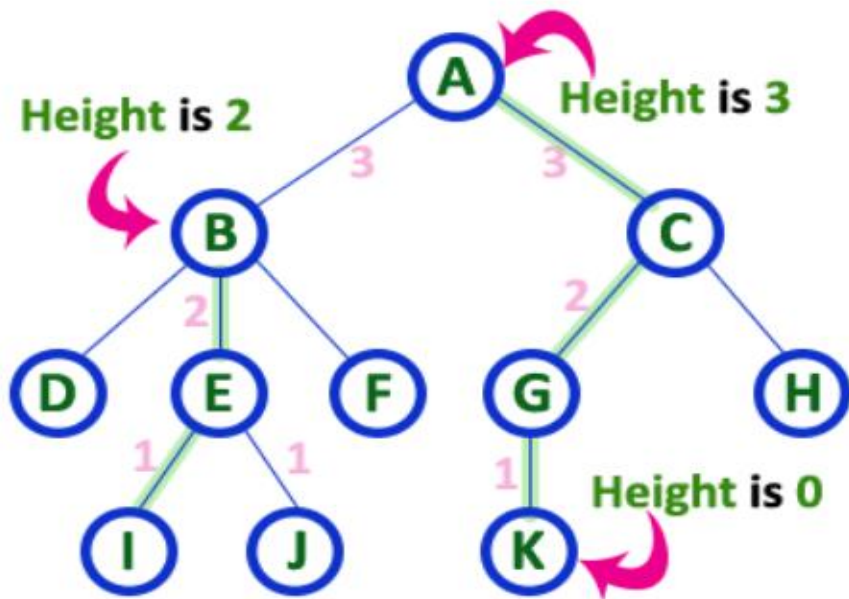
Terminologies In Tree Data Structure

level of a tree



Terminologies In Tree Data Structure

10. Height: the total number of edges from leaf node to a particular node in the longest path. In a tree, **height** of the **root** node is said to be **height of the tree**. In a tree, height of all **leaf nodes** is '0'.

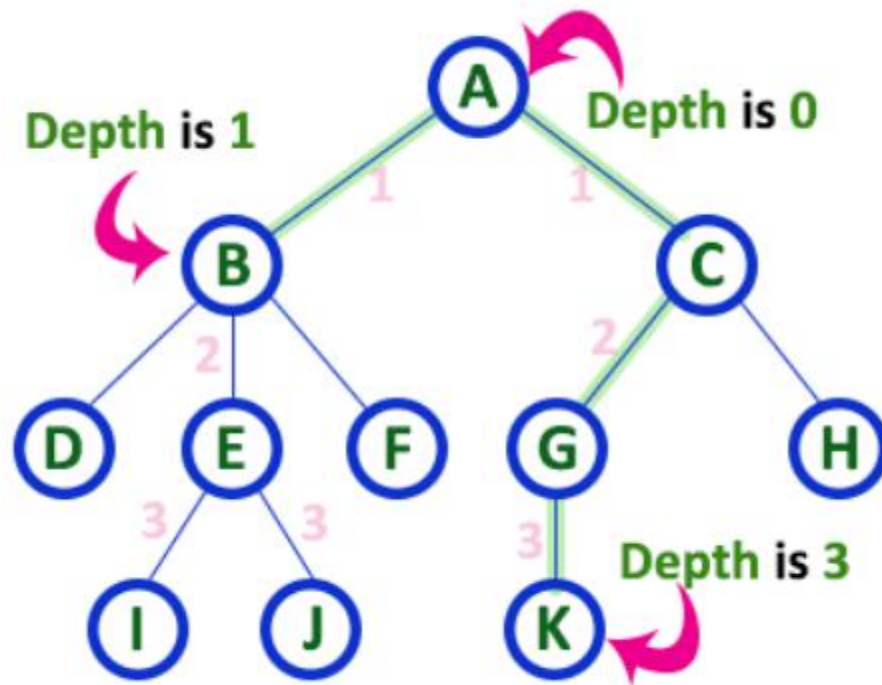


Here Height of tree is 3

- In any tree, 'Height of Node' is total number of Edges from leaf to that node in longest path.
- In any tree, 'Height of Tree' is the height of the root node.

Terminologies In Tree Data Structure

11. Depth : the total number of edges from root node to a particular node . In a tree, depth of the **root** node is '0'.

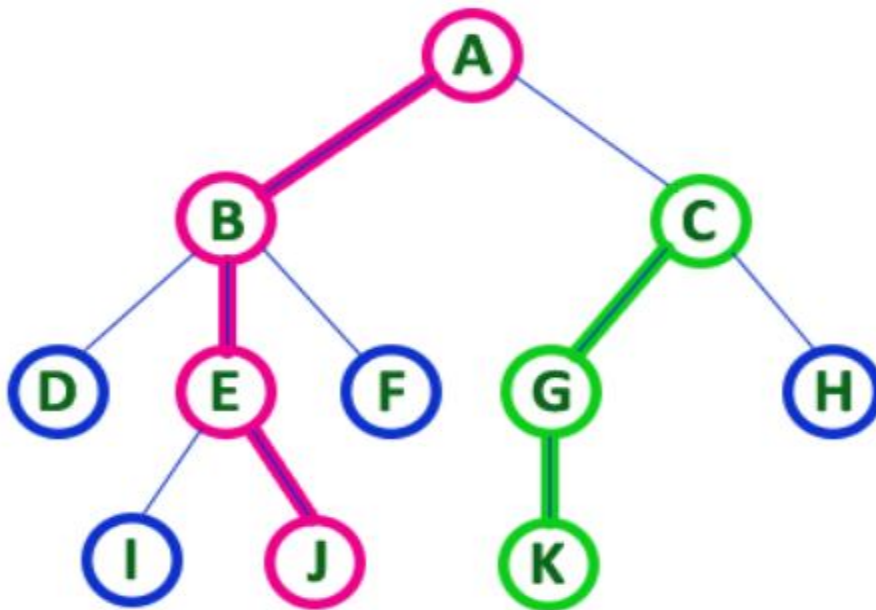


Here Depth of tree is 3

- In any tree, 'Depth of Node' is total number of Edges from root to that node.
- In any tree, 'Depth of Tree' is total number of edges from root to leaf in the longest path.

Terminologies In Tree Data Structure

12. Path :the sequence of Nodes and Edges from one node to another node is called as PATH between that two Nodes. Length of a Path is total number of nodes in that path. In below example the path **A - B - E - J** has length 4.



- In any tree, 'Path' is a sequence of nodes and edges between two nodes.

Here, 'Path' between A & J is

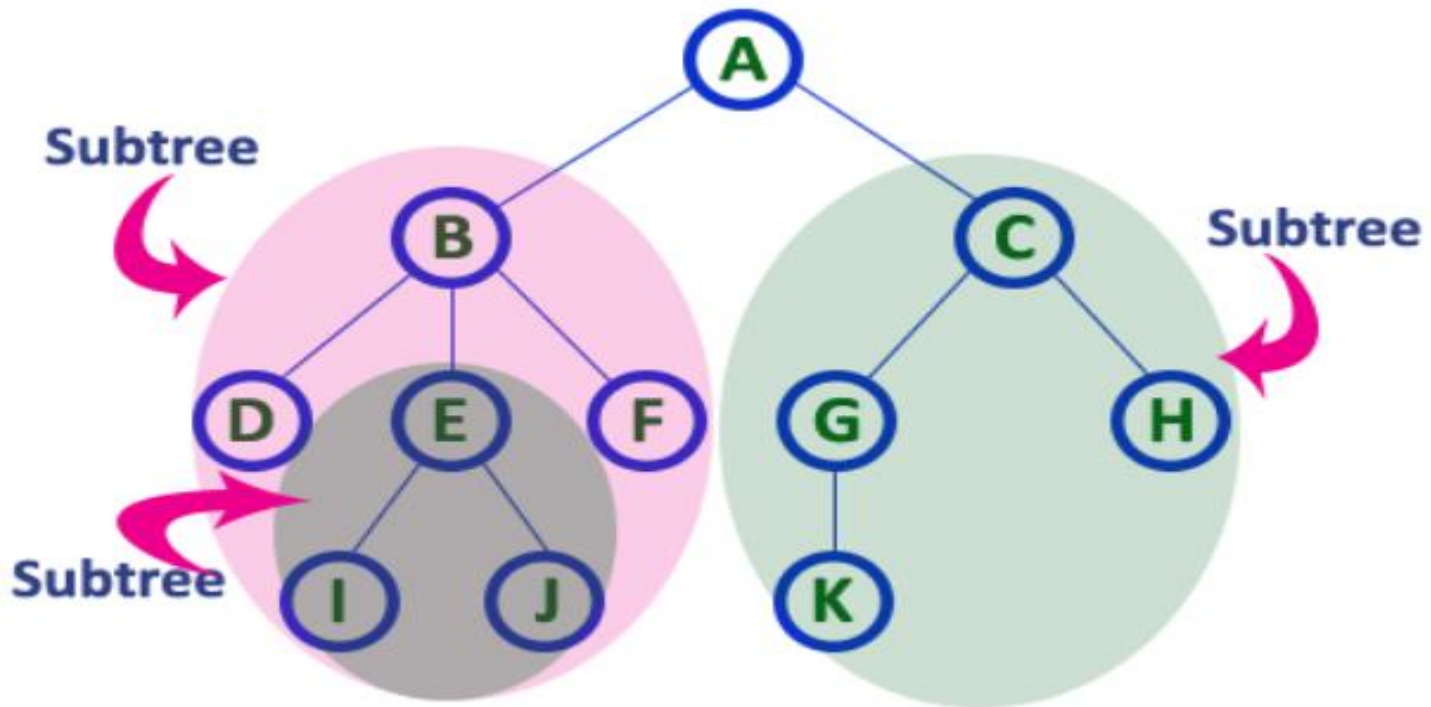
A - B - E - J

Here, 'Path' between C & K is

C - G - K

Terminologies In Tree Data Structure

13. Sub Tree :each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.



Properties of a Tree

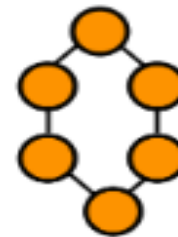
- The numbers of nodes in a tree must be a finite and nonempty set.
- There must exist a path to every node of a tree i.e., every node must be connected to some other node.
- There must not be any cycles in the tree. It means that the number of edges is one less than the number of nodes.



A tree



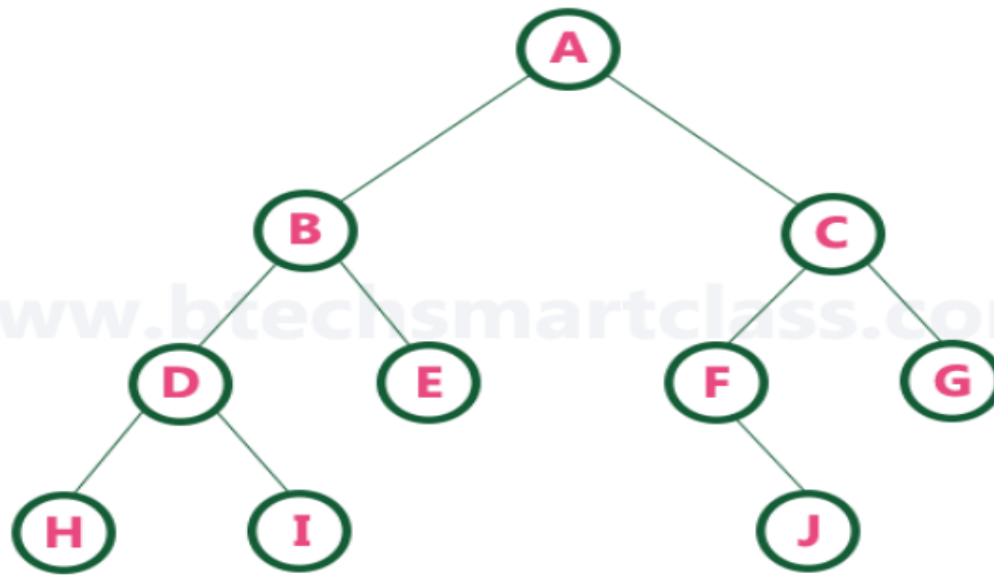
Not a tree
All nodes are not connected



Not a tree
Cycle exists

Binary Tree

- A tree in which every node can have a maximum of two children is called **Binary Tree**.
- Every node can have either **0 children or 1 child or 2 children but not more than 2 children**.



Types of Binary Trees

1. Full Binary Tree : every node has either **two or zero** number of children .

- In a **Full Binary** Tree, number of leaf nodes is the number of internal nodes plus 1

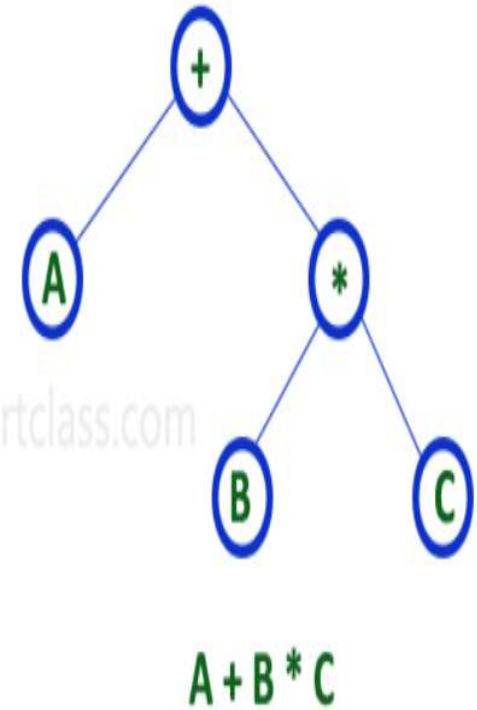
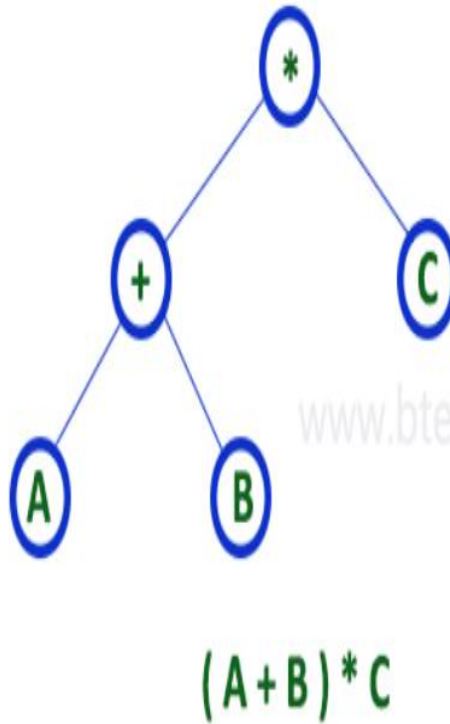
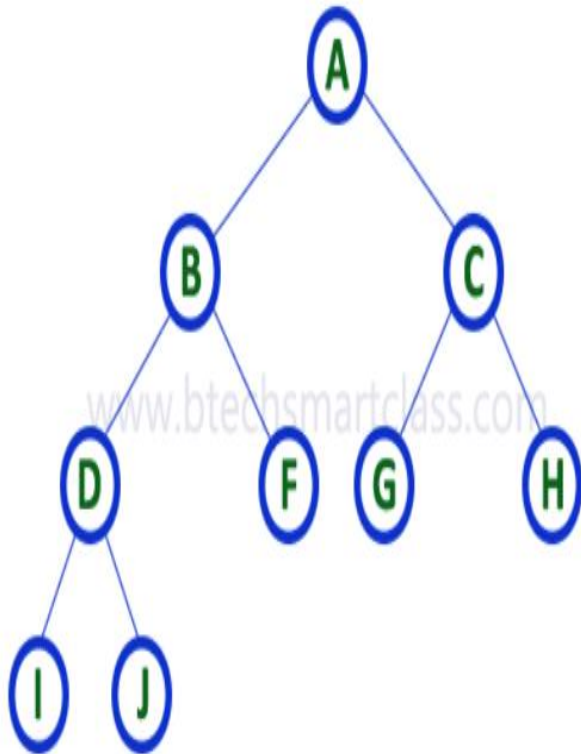
$$L = I + 1$$

Where **L = Number of leaf nodes,**

I = Number of internal node

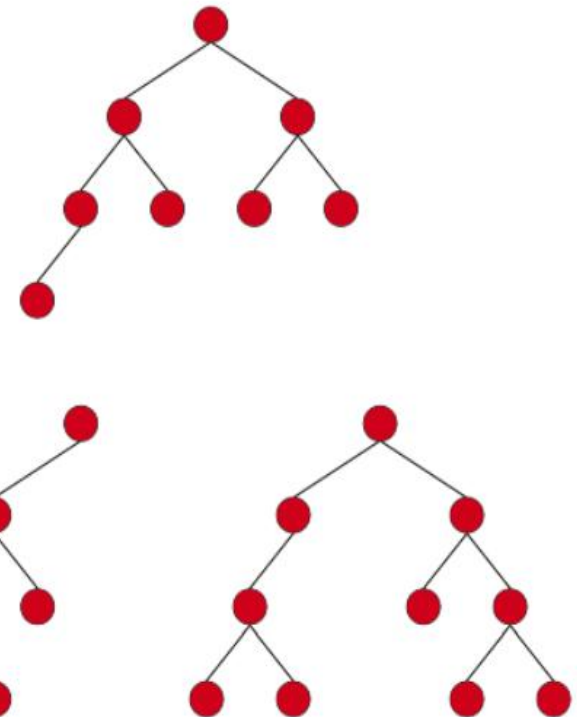
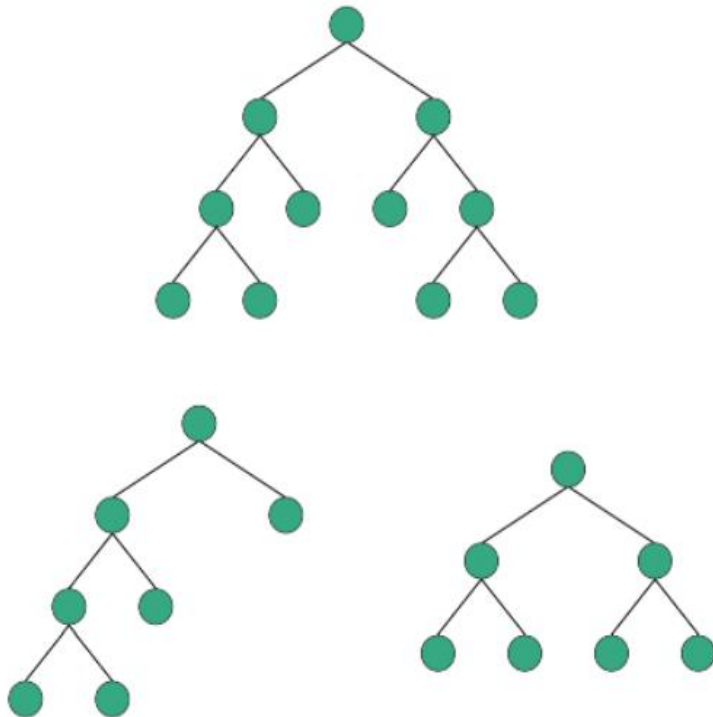
- used to represent mathematical expressions.

Full Binary Tree



Full Binary Tree

Valid and Invalid Structure of Full Binary Tree



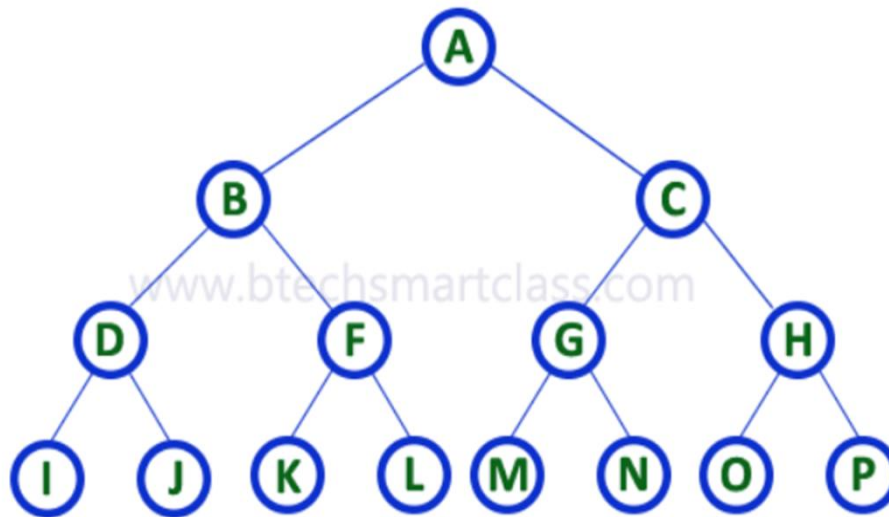
Types of Binary Trees

2. Complete Binary Tree :has all levels completely filled with nodes except the last level and in the last level, all the nodes are as left side as possible.

Valid and Invalid Structure of Complete Binary Tree

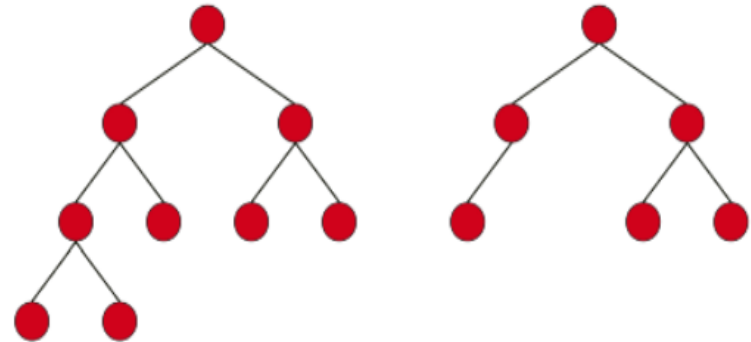
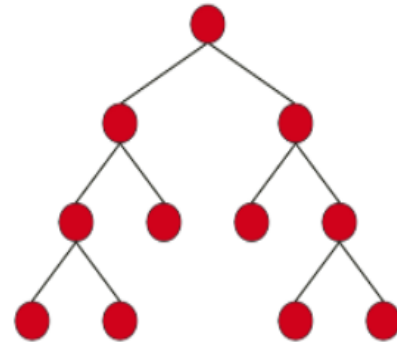
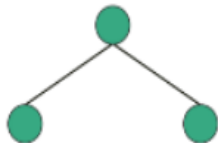
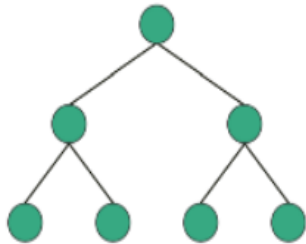
Types of Binary Trees

3. Perfect Binary Tree : all internal nodes have 2 children and all the leaf nodes are at the same depth or same level.



Perfect Binary Tree

Valid and Invalid Structure of Perfect Binary Tree

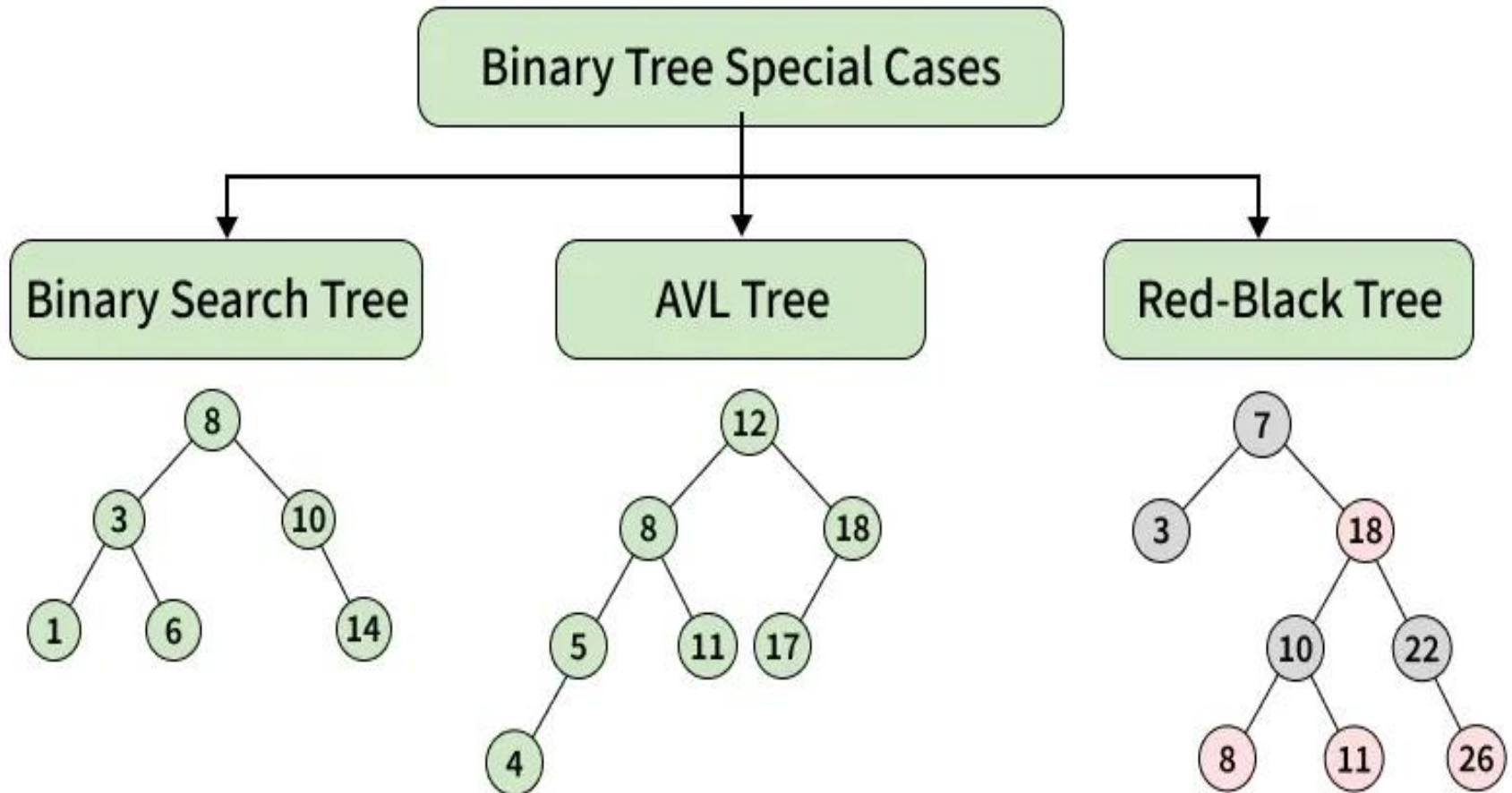


Some Special Types of Trees

On the basis of node values, the Binary Tree can be classified into the following special types:

- **Binary Search Tree**
- **AVL Tree**
- **Red Black Tree**
- **B Tree**
- **B+ Tree**

Binary Tree



Binary Tree Traversals

- When we wanted to display a binary tree, we need to follow some order in which all the nodes of that binary tree must be displayed.
- In any binary tree, displaying order of nodes depends on the traversal method .
- **Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal.**

Binary Tree Traversals

- **Tree traversal** refers to **the process of visiting or accessing** each node of a tree exactly once in a specific order.
- **Unlike linear** data structures such as **arrays, linked lists, or queues (which have only one logical way of traversal)**, **trees offer multiple** ways to traverse their nodes.

There are three **types of binary tree traversals**:

1. **In -Order Traversal- (leftChild - root -rightChild)**
2. **Pre - Order Traversal - (root - leftChild - rightChild)**
3. **Post - Order Traversal- (leftChild - rightChild - root)**

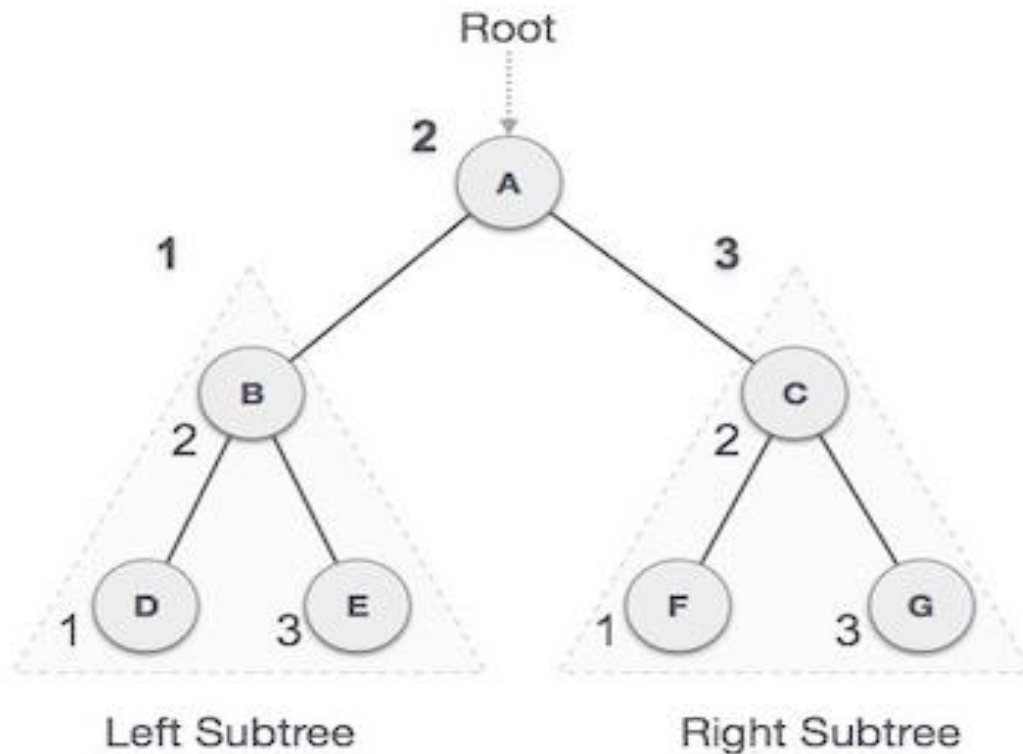
In - Order Traversal

- In **In-Order traversal**, the **root** node is visited between the **left** child and **right** child.
- The **left** child node is visited **first**, then the **root** node is visited and later we go for visiting the **right** child node.
- This **in-order traversal** is applicable for every root node of all subtrees in the tree.
- This is performed **recursively** for all nodes in the tree.

In - Order Traversal

The output of in-order traversal of this tree will be –

D → B → E → A → F → C → G



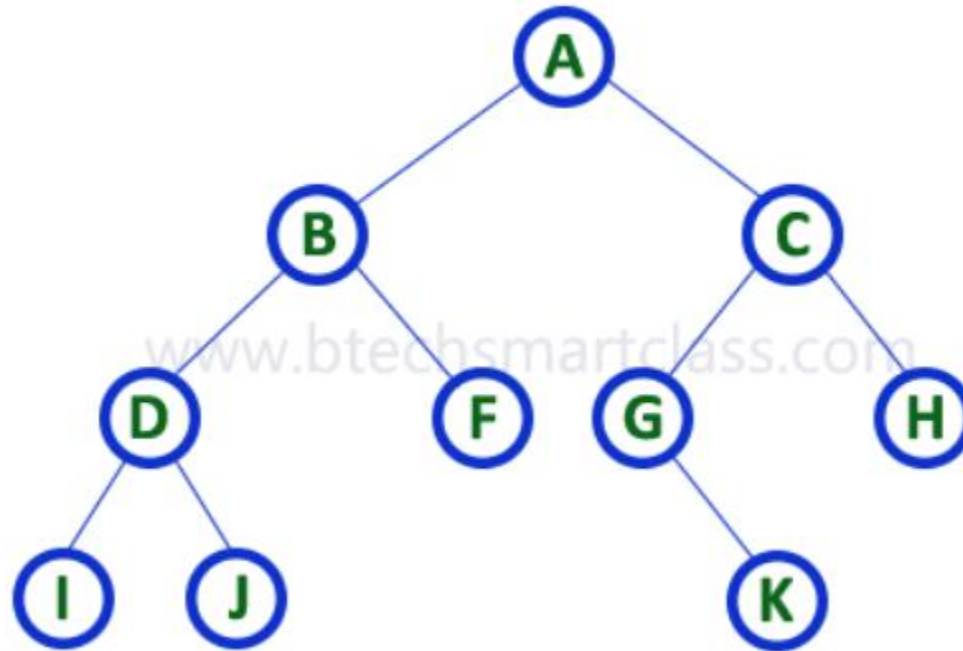
In - Order Traversal Algorithm

Until all nodes are traversed :

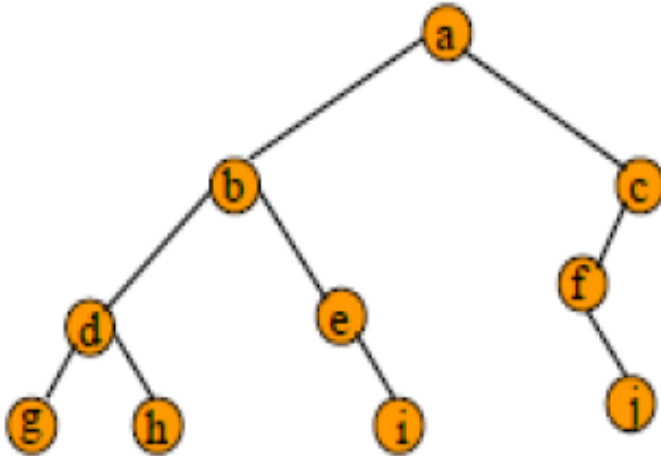
- **Step 1** : Recursively traverse **left subtree**.
- **Step 2**: Visit **root** node.
- **Step 3** : Recursively traverse right **subtree**.

In - Order Traversal

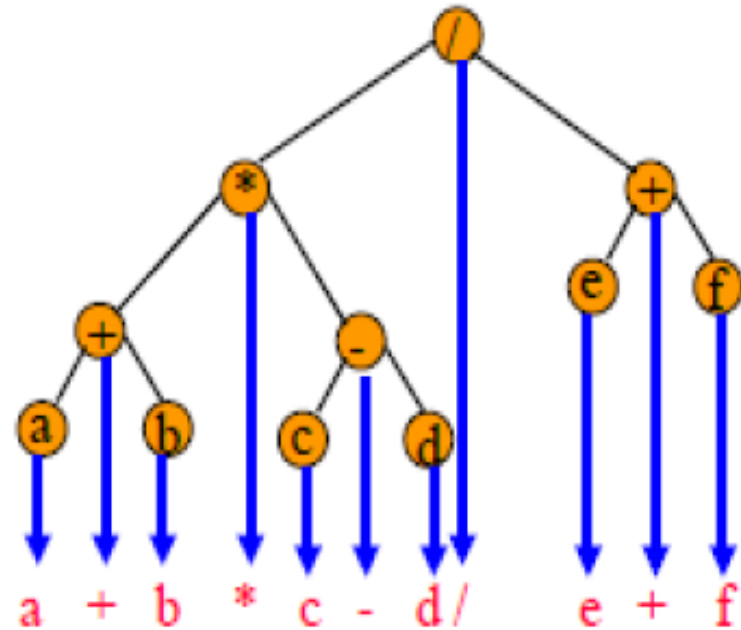
In-Order Traversal for above example of binary tree is : **I - D - J - B - F - A - G - K - C - H**



Example of Tree Traversal



g d h b e i a f j c



Inorder traversal

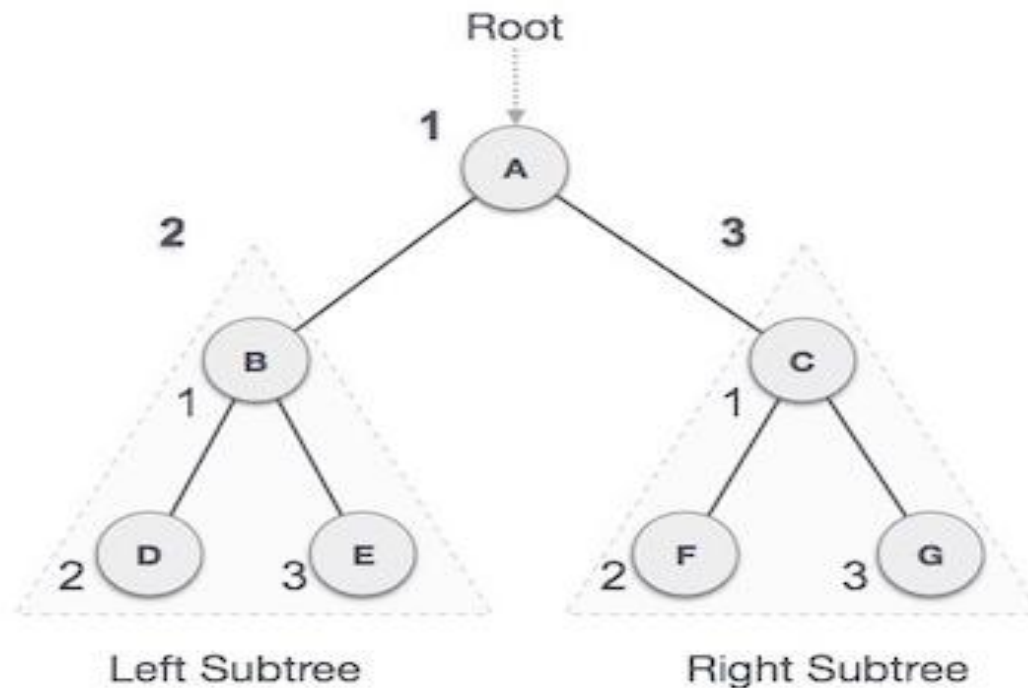
Pre - Order Traversal

- In **Pre-Order traversal**, the **root** node is visited **before** the **left** child and **right** child nodes.
- The **root** node is visited **first**, then its **left** child and later its **right** child.
- This **pre-order traversal** is applicable for every root node of all subtrees in the tree.

Pre - Order Traversal

- The output of pre-order traversal of this tree will be :

A → B → D → E → C → F → G



Pre - Order Traversal Algorithm

Until all nodes are traversed :

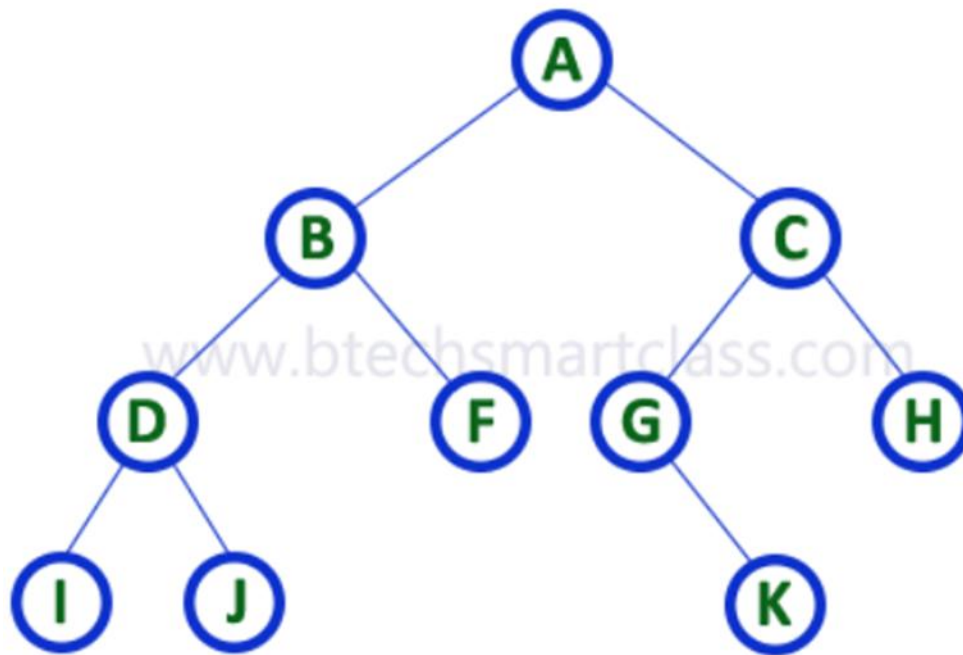
Step 1: Visit **root** node.

Step 2: Recursively traverse **left subtree**.

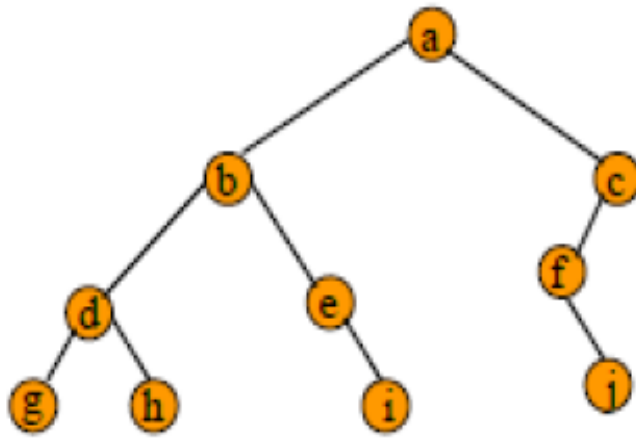
Step 3 : Recursively traverse **right subtree**.

Pre - Order Traversal

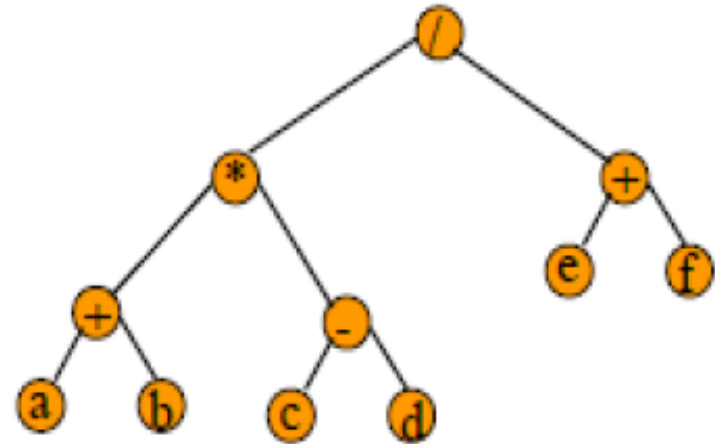
Pre-Order Traversal for above example binary tree is :**A - B - D - I - J - F - C - G - K - H**



Example of Tree Traversal



a b d g h e i c f j



/ * + a b - c d + e f

Preorder traversal

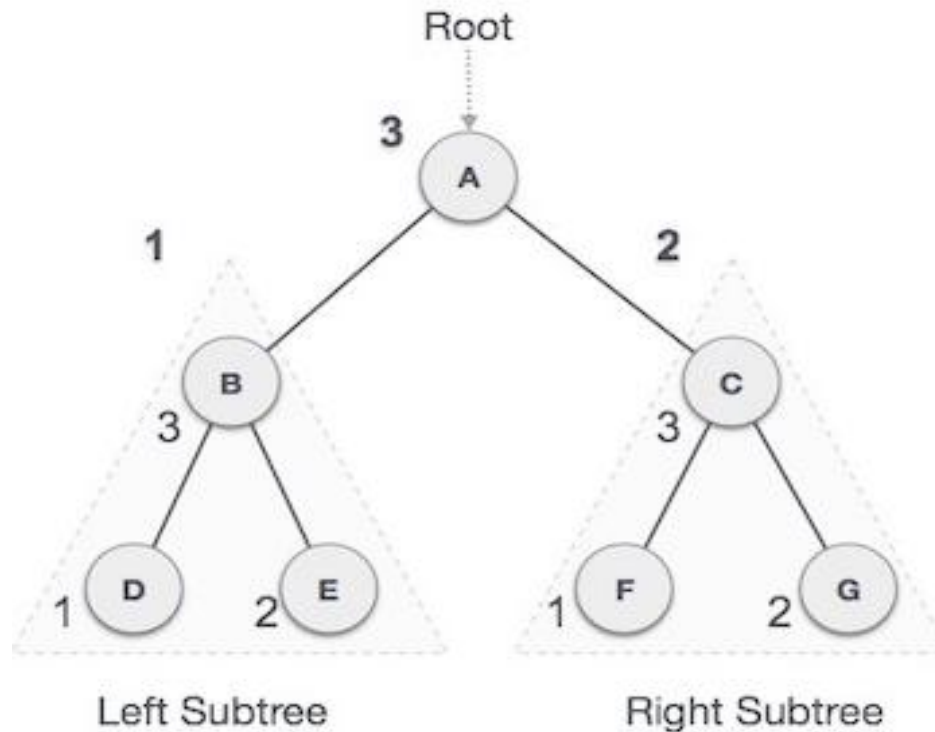
Post - Order Traversal

- In Post-Order traversal, the **root** node is visited **after left** child and **right** child.
- In this traversal, **left** child node is visited **first**, then its **right** child and then its **root** node.
- This is recursively performed until the right most node is visited.

Post - Order Traversal

The output of post-order traversal of this tree will be –

D → E → B → F → G → C → A



Post - Order Traversal Algorithm

Until all nodes are traversed:

Step 1: Recursively traverse **left subtree**.

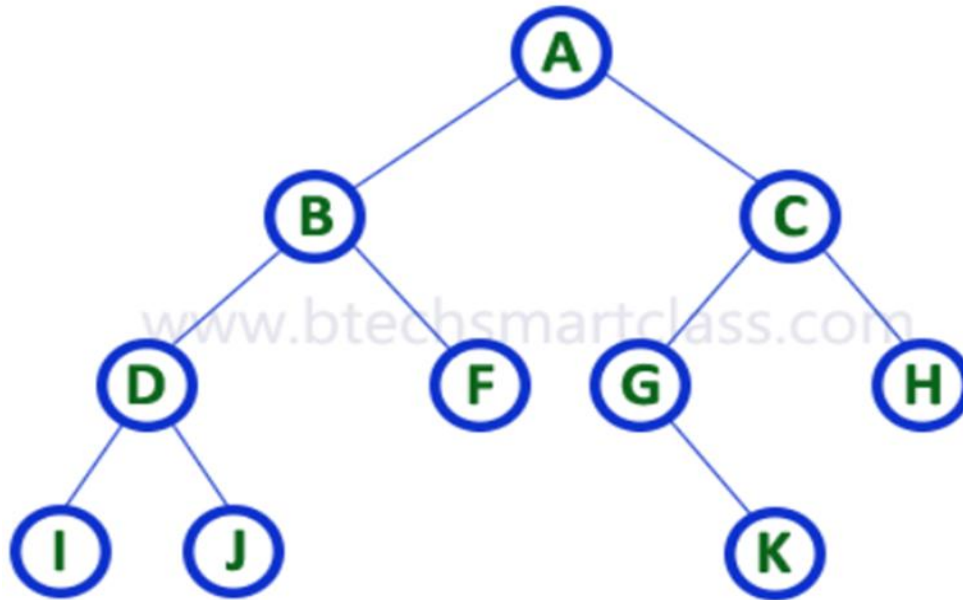
Step 2: Recursively traverse **right subtree**.

Step 3: Visit **root** node.

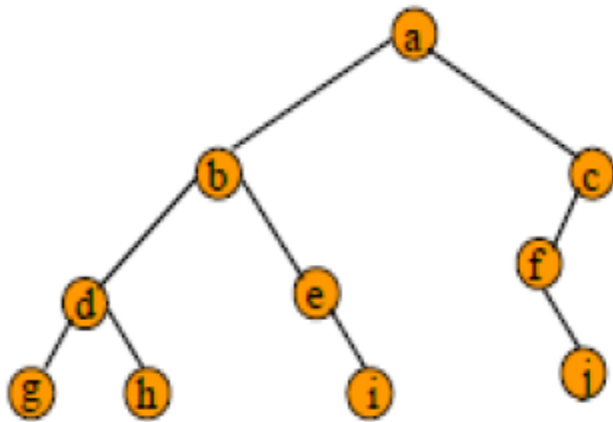
Post - Order Traversal

Here we have visited in the order of :

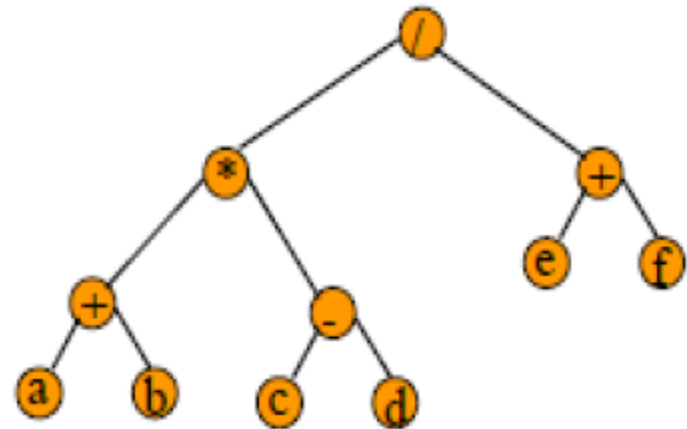
I - J - D - F - B - K - G - H - C - A using Post-Order Traversal.



Example of Tree Traversal



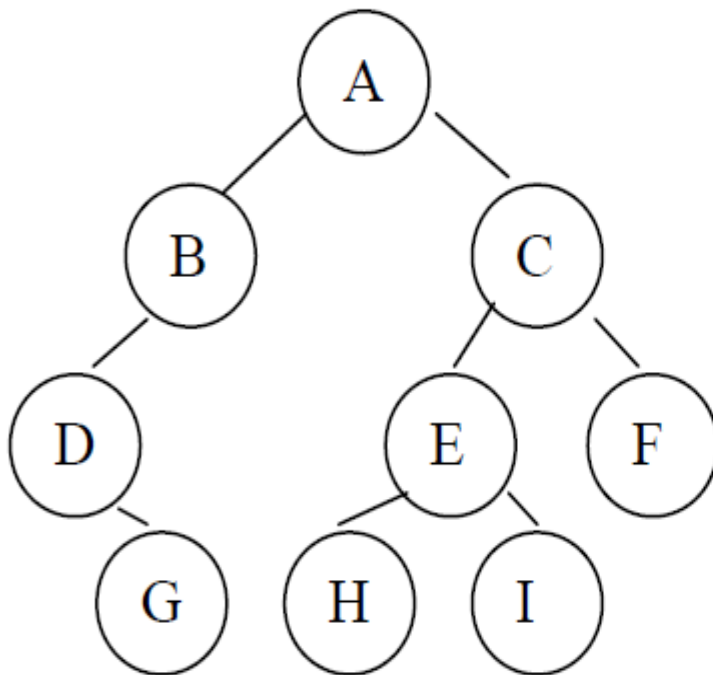
ghdiebjfca



ab+cd-*ef+ /

Postorder Traversal

Example of Tree Traversal



Preorder – ABDGCEHIF

Inorder – DGBAHEICF

Postorder – GDBHIEFCA